

BSTs and tree traversals

4/30/25

Administrivia

- HW 6 (Scoring and TestScoring) due tonight

Treasure hunting

WWWWWWWW

WS.T.TW

WT.....W

W...WWW

WWWWWWWW

Treasure hunting

```
WWWWWWWWW
WS . T . . TW
WT . . . . W
W . . . WWW
WWWWWWWWW
```

Suppose that the first treasure discovered is the closer of the two treasures to the right of the starting location. Which of the two remaining treasures **MUST** be discovered next? Briefly justify your answer. (Hint: What does finding that treasure first tell you about the order in which the program is trying the directions?)

Treasure hunting

```
WWWWWWWWW
WS . T . . TW
WT . . . . W
W . . . WWW
WWWWWWWWW
```

Suppose the first treasure discovered is the one below the starting location. Explain how either of the other treasures could be the next discovered. (Hint: For each of the remaining treasures, can you give an order in which the program could try directions that would result in that treasure being the next one discovered? Briefly justify that each of your chosen orders works.)

Recall: Set and Map

Set<K>

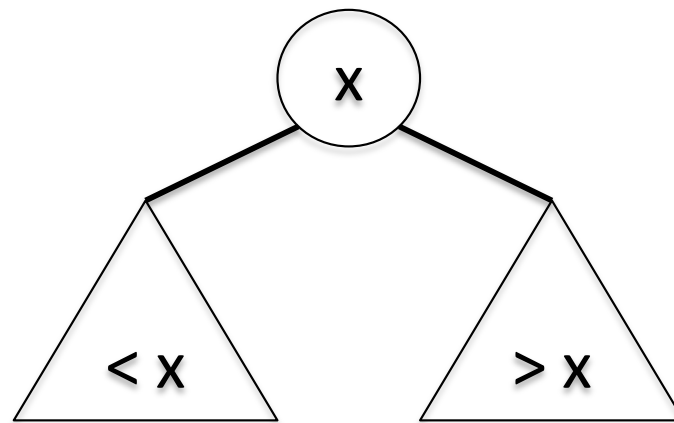
- Stores unordered collection of keys
- add
- contains
- iterator
- (size, remove, clear, etc)

Map<K,V>

- Stores key,value pairs; supports fast lookup based on key
- put(K,V)
- get(K)
- remove(K)
- contains(K)
- (keyset, values, isEmpty, size, clear, ...)

Recall: Binary Search Trees (BSTs)

- Tree with key stored at each node such that
 - Every node has 2 children (left and right)
 - Children can be null (nodes have 0, 1, 2 actual children)
 - All keys in a node's right subtree are greater than its key and all in the left subtree are less



Recall: contains in a BST

```
boolean contains(E key) {  
    Node curr = root;  
    while(curr != null) {  
        int cmp = key.compareTo(curr.key) ;  
        if(cmp < 0) curr = curr.left;  
        else if(cmp > 0) curr = curr.right;  
        else return true;  
    }  
    return false;  
}
```


Recursive implementation

```
boolean contains(E key) {  
    return contains_helper(root, key);  
}
```

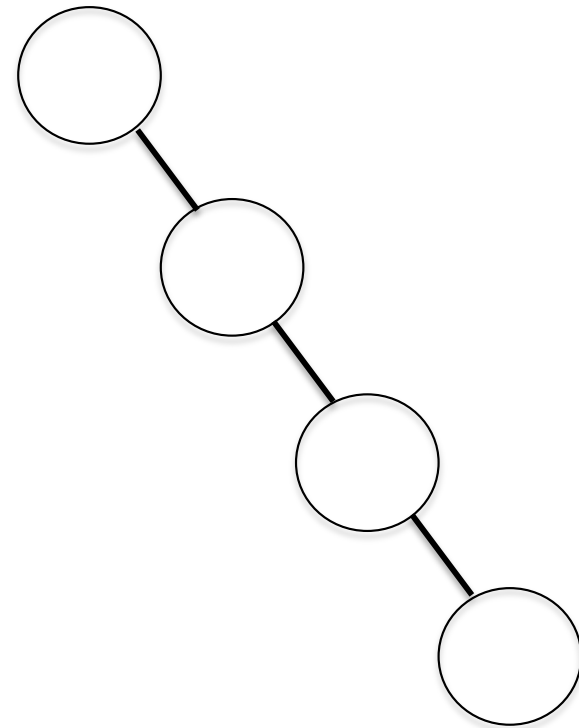
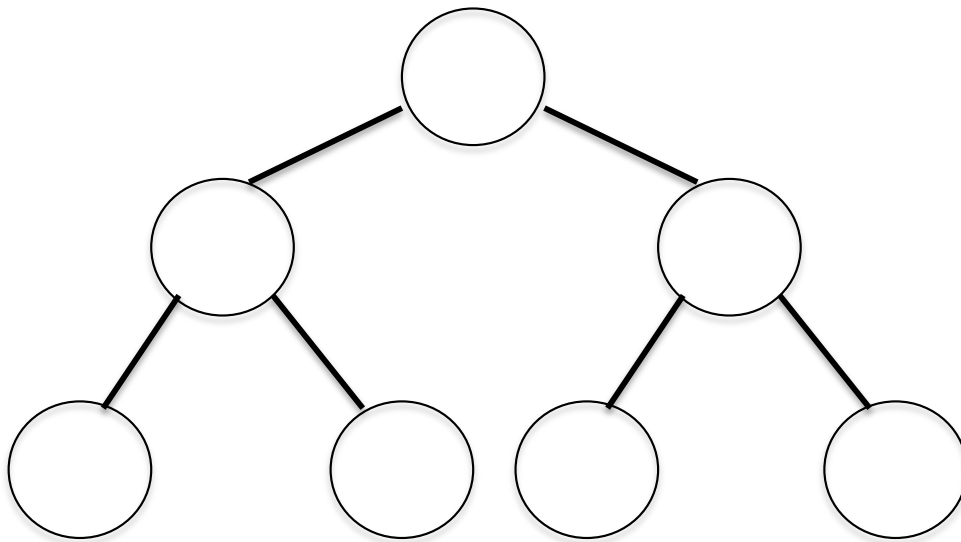
```
boolean contains_helper(Node curr, E key) {  
    if(curr == null) return false;  
    int cmp = key.compareTo(curr.key) ;  
    if(cmp < 0) return contains_helper(curr.left, key);  
    else if(cmp > 0) returns contains_helper(curr.right, key);  
    else return true;  
}
```

Other tree terminology

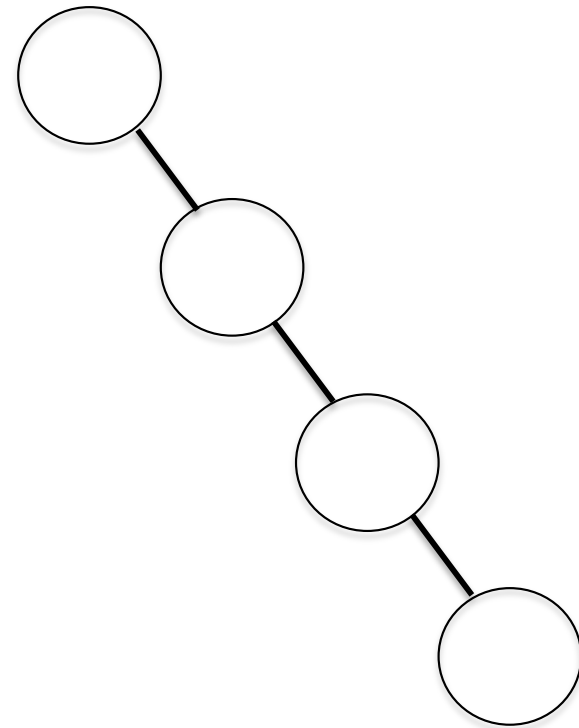
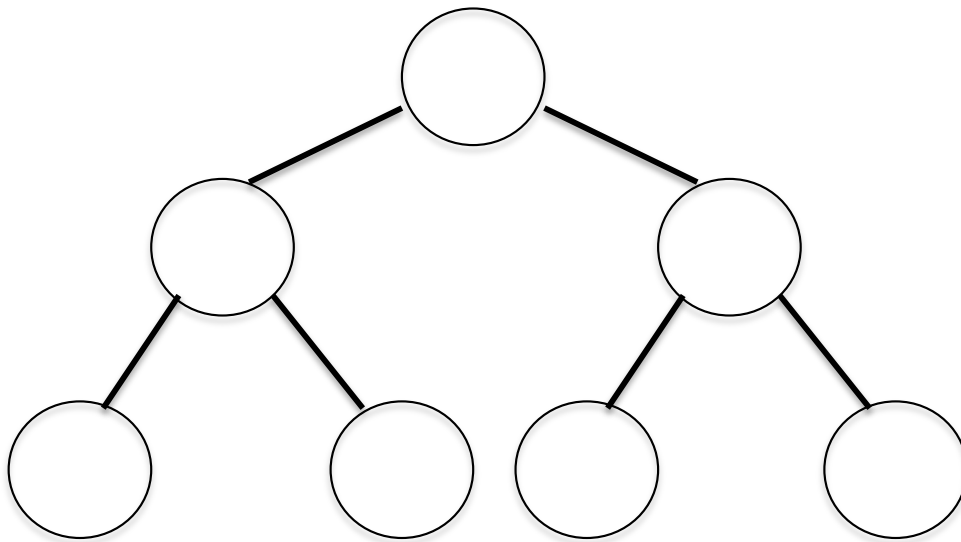
(book differs slightly; they count nodes)

- The *depth* of a node is the number of edges between it and the tree's root
- The *height* of a tree is the maximum number of edges from the root to a leaf

$O(\log n) \leq \text{height of } n\text{-node BST} \leq n-1$



$$O(\log n) \leq \text{height of } n\text{-node BST} \leq n-1$$



A tree is *balanced* if its height is $O(\log n)$

How can a BST implement add?

- Follow contains until curr is about to become a null reference (until “about to fall off”)
- Replace that null with the new node

How can a BST implement add?

- Follow contains until curr is about to become a null reference (until “about to fall off”)
- Replace that null with the new node

What is the running time of a BST add?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(\text{tree height})$
- D. $O(n)$
- E. None of the above

How can a BST implement remove?

- Begin by finding node with the key to remove
- Case 1: This node has 0 or 1 non-null children

How can a BST implement remove?

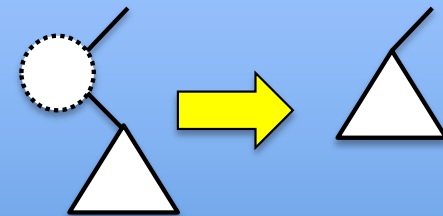
- Begin by finding node with the key to remove
- Case 1: This node has 0 or 1 non-null children
 - Remove node and replace it with child or null

How can a BST implement remove?

- Begin by finding node with the key to remove
- Case 1: This node has 0 or 1 non-null children
 - Remove node and replace it with child or null

How long does it take to move a subtree whose parent is removed?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. Not enough information
- E. None of the above



How can a BST implement remove?

- Begin by finding node with the key to remove
- Case 1: This node has 0 or 1 non-null children
 - Remove node and replace it with child or null
- Case 2: This node has 2 non-null children

How can a BST implement remove?

- Begin by finding node with the key to remove
- Case 1: This node has 0 or 1 non-null children
 - Remove node and replace it with child or null
- Case 2: This node has 2 non-null children
 - Replace the key with its *in-order successor* (next key in sorted order)
 - Remove the successor

How can we advance a Node reference to the next key?

- Case 1: Node has right subtree
 - Go one step right and then left as far as possible
- Case 2: Right subtree is null
 - Go up tree until reaching a node whose left child is the ancestor of your starting place
(requires parent references in the Node)
 - If you try to go up from the root, there is no next key

What is the running time of
remove in a balanced BST?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(\log^2 n) = O((\log n)^2)$
- D. $O(n)$
- E. None of the above

Recall: Desired properties

- Ability to quickly “jump” later in the sequence of keys (like binary search)
- Ability to easily add keys into the appropriate position (like linked lists)